

CSE563 — Assignment 4

Unit IV: Graph Theory

Maps to CO4 (analyse different graph theory concepts and use them in solving practical problems).
Marks: 50. Attempt all questions. Where an algorithm is traced, show the table of every step.

Part A — Short Answer and Definitions (10 marks, 2 each)

Q1. [2 marks] State the handshaking lemma. Hence determine whether a graph can exist with degree sequence $(5, 5, 4, 3, 2, 1)$, and whether one can exist with $(4, 4, 4, 4, 4)$.

Solution. Handshaking lemma: $\sum_{v \in V} \deg(v) = 2|E|$; consequently the number of odd-degree vertices is even.

$(5, 5, 4, 3, 2, 1)$: sum = 20, even, so $|E| = 10$; the odd-degree vertices are 5, 5, 3, 1 — four of them, an even count. **No contradiction**, such a graph can exist (max degree 5 is attainable with 6 vertices).

$(4, 4, 4, 4, 4)$: sum = 20, $|E| = 10$; this is K_5 , which is 4-regular. **Exists.**

Q2. [2 marks] Give the condition for a connected graph to have an Eulerian circuit and for it to have an Eulerian path. Why is there no comparably simple test for a Hamiltonian cycle?

Solution. Eulerian circuit: connected and *every* vertex has even degree. **Eulerian path** (open trail using every edge): connected and *exactly two* vertices have odd degree (the path starts at one and ends at the other).

Eulerian conditions are *local* — a property of individual degrees — so they are checkable in $O(n + m)$. Hamiltonicity is a *global* property of how vertices connect, and deciding it is **NP-complete**: no polynomial-time characterisation is known (and none exists unless $P = NP$). Only sufficient conditions exist, e.g. Dirac's ($\deg v \geq n/2$ for all v).

Q3. [2 marks] Prove that a tree with n vertices has exactly $n - 1$ edges.

Solution. By induction on n .

Basis ($n = 1$): a single vertex, no edges: $0 = 1 - 1$. ✓

Inductive step: assume every tree with k vertices has $k - 1$ edges. Let T be a tree with $k + 1$ vertices. Every finite tree has at least one *leaf* (a vertex of degree 1) — follow any longest path and its endpoint cannot continue without creating a cycle. Remove a leaf v and its single incident edge: the result is still connected (no path through v was needed, as v had degree 1) and still acyclic, hence a tree on k vertices, which by hypothesis has $k - 1$ edges. Restoring v adds one edge, giving k edges for $k + 1$ vertices. ■

Q4. [2 marks] Explain why Dijkstra's algorithm fails on graphs with negative edge weights. Give a small counterexample.

Solution. Dijkstra is greedy: when it extracts the vertex with the smallest tentative distance it declares that distance **final**, on the assumption that no later path can be shorter — valid only because adding non-negative edges can never decrease a path's cost.

Counterexample: vertices s, a, b with $s \rightarrow a$ (weight 1), $s \rightarrow b$ (weight 5), $b \rightarrow a$ (weight -10). Dijkstra extracts a first (distance 1) and fixes $d[a] = 1$. But the true shortest path is $s \rightarrow b \rightarrow a$ with cost $5 - 10 = -5$. The answer is wrong.
Use **Bellman–Ford** ($O(nm)$) instead, which also detects negative cycles.

Q5. [2 marks] Define the chromatic number. State the bounds $\omega(G) \leq \chi(G) \leq \Delta(G) + 1$ and explain in one line why each holds.

Solution. $\chi(G)$ is the minimum number of colours in a proper vertex colouring (adjacent vertices get different colours).

Lower bound $\chi \geq \omega$: a clique of size ω has every pair adjacent, so its vertices need ω distinct colours among themselves.

Upper bound $\chi \leq \Delta + 1$: colour greedily in any order — when a vertex is reached, at most Δ neighbours are already coloured, so one of $\Delta + 1$ colours is always free.

Part B — Algorithm Tracing (24 marks)

Q6. [6 marks] For the graph with adjacency lists 1:[2, 3, 4], 2:[1, 5], 3:[1, 5, 6], 4:[1, 6], 5:[2, 3, 7], 6:[3, 4, 7], 7:[5, 6]: (a) Perform BFS from vertex 1, giving the visit order and the distance of every vertex. (b) Perform DFS from vertex 1 (recursive, neighbours in the listed order), giving the visit order. (c) List the BFS tree edges and state which BFS property they demonstrate. (d) Identify one back edge found by the DFS and state what it proves.

Solution. (a) BFS from 1.

Dequeue	Enqueued	Queue after
1	2, 3, 4	[2, 3, 4]
2	5	[3, 4, 5]
3	6 (5 already seen)	[4, 5, 6]
4	— (6 already seen)	[5, 6]
5	7	[6, 7]
6	—	[7]
7	—	[]

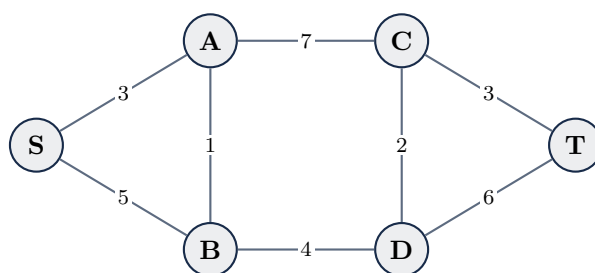
Visit order: 1, 2, 3, 4, 5, 6, 7. Distances: $d(1) = 0$; $d(2) = d(3) = d(4) = 1$; $d(5) = d(6) = 2$; $d(7) = 3$.

(b) DFS from 1. $1 \rightarrow 2 \rightarrow 5 \rightarrow 3$ (5's neighbours: 2 seen, then 3) $\rightarrow 6$ (3's: 1 seen, 5 seen, then 6) $\rightarrow 4$ (6's: 3 seen, then 4); 4's neighbours 1 and 6 both seen, backtrack to 6 $\rightarrow 7$. **Visit order:** 1, 2, 5, 3, 6, 4, 7.

(c) BFS tree edges: 1-2, 1-3, 1-4, 2-5, 3-6, 5-7 — six edges for seven vertices, so it is a spanning tree. They demonstrate that **BFS finds shortest paths in an unweighted graph**: the tree path from 1 to any vertex has length exactly equal to that vertex's distance (e.g. 1-2-5-7 has length $3 = d(7)$).

(d) Back edge: during the DFS, when at vertex 3 the neighbour 1 is already visited and is an *ancestor* of 3 in the DFS tree ($1 \rightarrow 2 \rightarrow 5 \rightarrow 3$), so 3-1 is a back edge. It proves the graph contains a **cycle**: $1 \rightarrow 2 \rightarrow 5 \rightarrow 3 \rightarrow 1$. (Similarly 4-1 and 7-5.)

Q7. [7 marks] Apply Dijkstra's algorithm from vertex S to the weighted graph below. Show the full table of tentative distances after each extraction, and state the shortest path and its cost to T .



Solution.

Extract	<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>T</i>	Relaxations performed
init	0	∞	∞	∞	∞	∞	—
<i>S</i> (0)	0	3	5	∞	∞	∞	<i>A</i> : 0+3; <i>B</i> : 0+5
<i>A</i> (3)	0	3	4	10	∞	∞	<i>B</i> : 3+1 = 4 < 5 ✓; <i>C</i> : 3+7
<i>B</i> (4)	0	3	4	10	8	∞	<i>D</i> : 4+4 = 8
<i>D</i> (8)	0	3	4	10	8	14	<i>C</i> : 8+2 = 10 < 10; <i>T</i> : 8+6 = 14
<i>C</i> (10)	0	3	4	10	8	13	<i>T</i> : 10+3 = 13 < 14 ✓
<i>T</i> (13)	0	3	4	10	8	13	—

Final distances: $S=0, A=3, B=4, D=8, C=10, T=13$.

Shortest path to T : trace parents backwards — T was finalised from C , C from A , A from S :

$$S \rightarrow A \rightarrow C \rightarrow T, \quad \text{cost } 3 + 7 + 3 = 13.$$

Note that B 's distance improved from 5 to 4 *after* A was extracted — an illustration of why relaxation must continue even for vertices that already have a finite label.

Q8. [5 marks] For the same graph as the previous question, find a minimum spanning tree using Kruskal's algorithm. List the edges in the order they are considered, mark those rejected and why, and give the total weight. Then state whether the MST equals Dijkstra's shortest-path tree, with justification.

Solution. Edges sorted by weight: AB 1, CD 2, SA 3, CT 3, BD 4, SB 5, DT 6, AC 7.

Edge (weight)	Action	Reason
AB (1)	accept	components $\{A, B\}$
CD (2)	accept	$\{C, D\}$
SA (3)	accept	joins S to $\{A, B\}$
CT (3)	accept	joins T to $\{C, D\}$
BD (4)	accept	joins the two components; now all 6 vertices connected
SB (5)	reject	would create cycle S - A - B - S
DT (6)	reject	cycle (D and T already connected via C)
AC (7)	reject	cycle; also we already have $n - 1 = 5$ edges

MST edges: AB, CD, SA, CT, BD . **Total weight** = $1 + 2 + 3 + 3 + 4 = 13$.

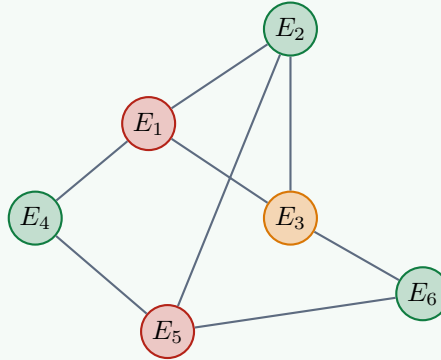
Comparison with the shortest-path tree. Dijkstra's tree from S (previous question) consists of the parent edges SA (3), AB (1), BD (4), AC (7), CT (3), of total weight **18**.

The two trees are therefore **different edge sets**: the MST takes CD (2) and omits AC (7), giving total weight 13 instead of 18. They optimise different quantities — the MST minimises the *sum of all edge weights*, while Dijkstra's tree minimises the *path length from the root to each individual vertex*, with no regard for the total.

A caution on this particular instance: the path S - A - B - D - C - T inside the MST costs $3 + 1 + 4 + 2 + 3 = 13$, which happens to equal the true shortest distance to T . That is a coincidence caused by the tie at C ($d[C] = 10$ via either A or D); in general the path between two vertices in an MST is **not** a shortest path, and no conclusion should be drawn from this coincidence.

Q9. [6 marks] Six examinations $E_1, \dots, E_6$ must be scheduled. Two exams conflict (cannot share a slot) if a student sits both. The conflicts are: $E_1-E_2, E_1-E_3, E_1-E_4, E_2-E_3, E_2-E_5, E_3-E_6, E_4-E_5, E_5-E_6$. (a) Draw the conflict graph and list the degree of each vertex. (b) Apply the Welsh–Powell algorithm and give the colour classes. (c) State the minimum number of slots, and prove that fewer are impossible. (d) Give the final timetable.

Solution. (a) **Conflict graph.**



Degrees: $E_1 = 3$ (to 2,3,4); $E_2 = 3$ (1,3,5); $E_3 = 3$ (1,2,6); $E_5 = 3$ (2,4,6); $E_4 = 2$ (1,5); $E_6 = 2$ (3,5). Sum = 16 = $2|E| = 2(8)$. ✓

(b) **Welsh–Powell.** Order by decreasing degree: $E_1, E_2, E_3, E_5, E_4, E_6$.

- **Colour 1 (red):** E_1 . Next in order not adjacent to anything red: E_2 ? adjacent to E_1 , skip. E_3 ? adjacent, skip. E_5 ? not adjacent to E_1 ✓ → red. E_4 ? adjacent to E_5 , skip. E_6 ? adjacent to E_5 , skip. **Red** = $\{E_1, E_5\}$.
- **Colour 2 (green):** E_2 (first uncoloured). E_3 ? adjacent to E_2 , skip. E_4 ? not adjacent to E_2 ✓ → green. E_6 ? not adjacent to E_2 or E_4 ✓ → green. **Green** = $\{E_2, E_4, E_6\}$.
- **Colour 3 (gold):** E_3 remains. **Gold** = $\{E_3\}$.

(c) **Minimum slots** = 3. Three colours suffice by (b). Fewer is impossible because E_1, E_2, E_3 are mutually adjacent — a **triangle** (K_3), so $\omega(G) = 3$ and $\chi(G) \geq \omega(G) = 3$. Hence $\chi(G) = 3$.

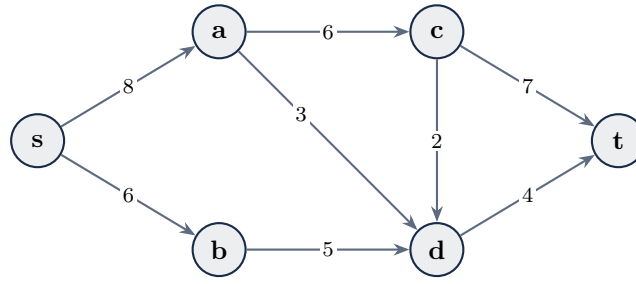
(d) **Timetable.**

Slot	Examinations
1	E_1, E_5
2	E_2, E_4, E_6
3	E_3

No two exams in a slot share an edge, so no student has a clash.

Part C — Network Flow (16 marks)

Q10. [10 marks] For the flow network below (source s , sink t , capacities shown): (a) Apply the Ford–Fulkerson method, listing each augmenting path with its bottleneck and the running flow value. (b) State the maximum flow. (c) Find the minimum cut (S, T) , list the arcs crossing it, and verify its capacity equals the max flow. (d) Explain why arcs from T back into S are not counted.



Solution. (a) **Augmenting paths.**

#	Path	Bottleneck	Flow so far
1	$s \rightarrow a \rightarrow c \rightarrow t$	$\min(8, 6, 7) = 6$	6
2	$s \rightarrow a \rightarrow d \rightarrow t$	$\min(2, 3, 4) = 2$	8
3	$s \rightarrow b \rightarrow d \rightarrow t$	$\min(6, 5, 2) = 2$	10

After path 1: sa has 2 left, ac has 0, ct has 1. After path 2: sa has 0, ad has 1, dt has 2. After path 3: sb has 4, bd has 3, dt has 0.

Final flows: $f(s, a) = 8$, $f(s, b) = 2$, $f(a, c) = 6$, $f(a, d) = 2$, $f(b, d) = 2$, $f(c, t) = 6$, $f(d, t) = 4$, $f(c, d) = 0$. Conservation at a : in 8, out $6 + 2$ ✓; at d : in $2 + 2$, out 4 ✓.

Residual search for a further augmenting path. Starting from s :

- $s \rightarrow a$: saturated (residual 0).
- $s \rightarrow b$: residual $6 - 2 = 4$, so b is reachable.
- From b : $b \rightarrow d$ residual $5 - 2 = 3$, so d is reachable.
- From d : $d \rightarrow t$ is saturated (residual 0). The *reverse* arc $d \rightarrow a$ has residual $f(a, d) = 2$, so a is reachable after all.
- From a : $a \rightarrow c$ is saturated (residual 0); the reverse arc $a \rightarrow s$ goes back to a visited vertex.

No path reaches t . **No augmenting path remains, so the flow is maximum.**

(b) Maximum flow = 10. (Sanity check against capacity into t : $c(c, t) + c(d, t) = 7 + 4 = 11$, but c can receive at most $c(a, c) = 6$, so at most $6 + 4 = 10$ can arrive. ✓)

(c) Minimum cut. The set reachable from s in the final residual network, as traced above, is

$$S = \{s, a, b, d\}, \quad T = \{c, t\}.$$

Arcs crossing from S to T : $a \rightarrow c$ (capacity 6) and $d \rightarrow t$ (capacity 4). The arc $c \rightarrow d$ runs from T back into S and is not counted; $c \rightarrow t$ lies wholly inside T ; $s \rightarrow a$, $s \rightarrow b$, $a \rightarrow d$, $b \rightarrow d$ lie wholly inside S .

$$c(S, T) = 6 + 4 = \mathbf{10} = \text{max flow. } \checkmark$$

Both crossing arcs are saturated and the single $T \rightarrow S$ arc carries zero flow — the signature of a minimum cut.

(d) A cut's capacity measures the maximum that can be *delivered* from S to T . Arcs directed from T back into S carry flow the wrong way across the frontier; they can only *reduce* the net transfer, never increase the deliverable amount, so they are excluded from the capacity. Formally, $|f| = f(S, T) - f(T, S) \leq c(S, T)$, and equality at the maximum requires every $S \rightarrow T$ arc saturated and every $T \rightarrow S$ arc carrying zero flow — which is exactly the case here ($f(c, d) = 0$).

Q11. [6 marks] A company must assign four engineers $\{E_1, E_2, E_3, E_4\}$ to four projects $\{P_1, P_2, P_3, P_4\}$, one each. Engineer E_1 is qualified for P_1, P_2 ; E_2 for P_2 ; E_3 for P_2, P_3 ; E_4 for P_3, P_4 . (a) Model this as a maximum bipartite matching problem and explain how to convert it into a max-flow problem. (b) Find the maximum matching and state whether a complete assignment is possible.

Solution. (a) Reduction to max flow. Build a directed network: add a source s with an arc of capacity 1 to each engineer, an arc of capacity 1 from engineer E_i to each project P_j they are qualified for, and an arc of capacity 1 from each project to a sink t .

The unit capacities on the $s \rightarrow E_i$ arcs force each engineer to be used at most once; those on the $P_j \rightarrow t$ arcs force each project to receive at most one engineer. Because all capacities are integers, the integrality theorem guarantees an integral maximum flow, and the saturated middle arcs are exactly a maximum matching. The **value of the max flow equals the size of the maximum matching**.

(b) Maximum matching. Augmenting paths:

1. $s \rightarrow E_1 \rightarrow P_1 \rightarrow t$ (bottleneck 1). Match E_1-P_1 .
2. $s \rightarrow E_2 \rightarrow P_2 \rightarrow t$. Match E_2-P_2 .
3. $s \rightarrow E_3 \rightarrow P_3 \rightarrow t$. Match E_3-P_3 .
4. $s \rightarrow E_4 \rightarrow P_4 \rightarrow t$. Match E_4-P_4 .

Flow value = 4.

Matching: E_1-P_1 , E_2-P_2 , E_3-P_3 , E_4-P_4 .

All four engineers are assigned, so **a complete (perfect) assignment is possible**.

Note on the ordering: the greedy order matters. Had E_1 been matched to P_2 first, E_2 (whose only option is P_2) would be blocked — but the max-flow algorithm recovers via the reverse arc $P_2 \rightarrow E_1$, rerouting E_1 to P_1 . This ability to *undo* an earlier choice is exactly what the residual reverse arcs provide.